

Manuale Teorico

Progetto 25 — ELM con Regularizzazione ℓ_1

Corso: Computational Mathematics for Learning and Data Analysis

A.A. 2025/2026

Abstract

Questo manuale raccoglie gli argomenti teorici necessari per lo sviluppo e la comprensione del **Progetto 25** del corso CM. Il progetto riguarda il training di una *Extreme Learning Machine* (ELM) con regularizzazione ℓ_1 (LASSO), risolto tramite due algoritmi: **A1 = IRLS** (Iteratively Reweighted Least Squares) e **A2 = Metodo del Sottogradient Deflesso con Target Level**. Il documento è strutturato come percorso di studio progressivo: si parte dal problema di minimi quadrati lineari, si analizzano i solutori numerici (fattorizzazione di Cholesky e gradiente coniugato), si introducono convessità e ottimalità per funzioni non differenziabili, e infine si descrivono i due algoritmi.

Basato sulle lezioni del corso: *Optimization* (OPT/3, OPT/4, OPT/5) e *Numerical Linear Algebra* (NLA/3, NLA/4, NLA/5, NLA/20).

Contents

1	Extreme Learning Machine e Formulazione del Problema	2
1.1	Architettura ELM	2
1.2	Problema di Ottimizzazione con Regularizzazione ℓ_1	2
2	Minimi Quadrati Lineari	2
2.1	Formulazione e Geometria	2
2.2	Rango Pieno per Colonne e Unicità	3
2.3	Equazioni Normali	3
2.4	Costo Computazionale	3
3	Fattorizzazione di Cholesky	3
3.1	Sfruttare la Simmetria: da LU a $LDL^\top$	3
3.2	Cholesky per Matrici SPD	4
4	Gradiente Coniugato (CG)	4
4.1	Motivazione: Sistemi Sparsi di Grandi Dimensioni	4
4.2	Riformulazione come Ottimizzazione	4
4.3	Spazi di Krylov e Ottimalità nel Sottospazio	5
4.4	Algoritmo CG e Q -Ortogonalità	5
4.5	Convergenza e Numero di Condizionamento	5
5	Convessità e Condizioni di Ottimalità	6
5.1	Definizioni di Convessità	6
5.2	Convergenza del Metodo del Gradiente con Forte Convessità	6
5.3	Condizioni di Ottimalità per Funzioni Lisce	6
5.4	Sottogradiente e Subdifferenziale	6
6	Ottimizzazione Non Liscia	7
6.1	Perché i Metodi Lisci Falliscono	7
6.2	Metodo del Sottogradiente con Passo DSS	7
6.3	Passo di Polyak	8
7	Algoritmo A2: Sottogradiente Deflesso con Target Level	8
7.1	Deflettere la Direzione	8
7.2	Scelta Ottimale di γ^i	8
7.3	Meccanismo Target Level	9
7.4	Algoritmo Completo (A2)	9
7.5	Note sulla Convergenza	10
8	Algoritmo A1: IRLS (Iteratively Reweighted Least Squares)	10
8.1	Motivazione: Approssimare ℓ_1 con Minimi Quadrati Pesati	10
8.2	Sottoproblema di Minimi Quadrati Pesato	10
8.3	Aggiornamento della Matrice Peso	10
8.4	Algoritmo Completo (A1)	11
8.5	Convergenza	11
9	Riepilogo: Connessioni tra gli Argomenti	11

1 Extreme Learning Machine e Formulazione del Problema

1.1 Architettura ELM

Una **Extreme Learning Machine** (ELM) è una rete neurale con un singolo strato nascosto in cui i pesi del primo strato W_1 (e i bias) sono *fissi e generati casualmente*, mentre solo i pesi di output $w \in \mathbb{R}^n$ vengono ottimizzati.

Dato un dataset di addestramento $\{(x_i, y_i)\}_{i=1}^m$ con $x_i \in \mathbb{R}^d$ e $y_i \in \mathbb{R}$, la rete calcola:

$$\hat{y}_i = \sum_{j=1}^n \sigma(W_{1,j} x_i + b_j) w_j,$$

dove σ è una funzione di attivazione non lineare (es. sigmoide o ReLU).

Rappresentazione matriciale

Si definisce la **matrice delle feature** $X \in \mathbb{R}^{m \times n}$:

$$X_{ij} = \sigma(W_{1,j} x_i + b_j).$$

Poiché W_1 è fissato, X è nota. Il vettore target è $y \in \mathbb{R}^m$, e l'output della rete si scrive compattamente come $\hat{y} = Xw$. Il training dell'ELM si riduce a un problema di **ottimizzazione lineare** nei pesi w .

1.2 Problema di Ottimizzazione con Regularizzazione ℓ_1

Addestrare l'ELM minimizzando l'errore quadratico con regularizzazione LASSO equivale a risolvere:

Problema Centrale del Progetto

$$\min_{w \in \mathbb{R}^n} f(w) = \underbrace{\|Xw - y\|_2^2}_{\text{errore quadratico}} + \lambda \underbrace{\|w\|_1}_{\text{regularizzazione } \ell_1}, \quad \lambda > 0.$$

Intuizione

La norma ℓ_1 **promuove la sparsità**: spinge molti pesi w_i ad essere *esattamente zero*, selezionando automaticamente le feature più rilevanti. Questo è particolarmente utile quando $n \gg m$ (molte feature, pochi campioni), scenario comune nelle ELM ad alto numero di neuroni nascosti. In contrasto, la regularizzazione ℓ_2 (Ridge) distribuisce uniformemente i pesi senza azzerarli.

Osservazione 1.1. La funzione obiettivo $f(w)$ è la somma di una parte *liscia* (il termine quadratico, differenziabile ovunque) e di una parte *non liscia* ($\lambda \|w\|_1$, non differenziabile nei punti con $w_i = 0$). Questa struttura mista richiede strumenti distinti: solutori lineari diretti/iterativi per il termine quadratico, e tecniche di ottimizzazione non liscia per la norma ℓ_1 .

2 Minimi Quadrati Lineari

2.1 Formulazione e Geometria

Dato $A \in \mathbb{R}^{m \times n}$ (con $m \geq n$) e $y \in \mathbb{R}^m$, il **problema dei minimi quadrati** è:

$$\min_{w \in \mathbb{R}^n} \|Aw - y\|_2.$$

Intuizione

Il vettore Aw vive nello spazio colonna $\text{Im}(A)$. Se $y \notin \text{Im}(A)$, non esiste soluzione esatta. Il problema chiede il punto in $\text{Im}(A)$ più vicino a y : tale punto è la **proiezione ortogonale** di y su $\text{Im}(A)$. Il vettore residuo $r = Aw_* - y$ è ortogonale a ogni colonna di A .

2.2 Rango Pieno per Colonne e Unicità

Definizione 2.1 (Rango Pieno per Colonne). $A \in \mathbb{R}^{m \times n}$ ha **rango pieno per colonne** se $\ker(A) = \{0\}$, ovvero $\text{rk}(A) = n$.

Teorema 2.1. Il problema dei minimi quadrati ha soluzione **unica** se e solo se A ha rango pieno per colonne. Inoltre, A ha rango pieno per colonne se e solo se $A^\top A$ è **definita positiva**.

Proof. Per ogni $z \neq 0$: $z^\top A^\top A z = \|Az\|^2 > 0 \iff Az \neq 0 \iff z \notin \ker(A)$. $\square$

2.3 Equazioni Normali

Minimizzare $\|Aw - y\|^2$ equivale a minimizzare:

$$f(w) = \frac{1}{2} w^\top A^\top A w - (A^\top y)^\top w + \frac{1}{2} \|y\|^2.$$

Il gradiente è $\nabla f(w) = A^\top A w - A^\top y$ e l'Hessiano è $A^\top A \succ 0$ (funzione strettamente convessa). Il minimo si trova ponendo il gradiente a zero:

Equazioni Normali

$$A^\top A w_* = A^\top y.$$

La matrice $Q = A^\top A$ è **simmetrica definita positiva** (SPD). La soluzione è $w_* = (A^\top A)^{-1} A^\top y =: A^+ y$, dove $A^+ = (A^\top A)^{-1} A^\top$ è la **pseudoinversa** di Moore-Penrose.

Osservazione 2.1. La denominazione “normali” viene dal fatto che il residuo $Aw_* - y$ è **normale** (perpendicolare) allo spazio $\text{Im}(A)$: $(Av)^\top (Aw_* - y) = 0$ per ogni $v \in \mathbb{R}^n$, che riscritta dà $A^\top (Aw_* - y) = 0$.

2.4 Costo Computazionale

Per $A \in \mathbb{R}^{m \times n}$ con $m > n$, sfruttando la simmetria di $A^\top A$:

1. Calcolo di $A^\top A$: costo mn^2 operazioni.
2. Calcolo di $A^\top y$: costo $2mn$ (trascurabile).
3. Risoluzione di $Qw = b$ con Cholesky: costo $\frac{1}{3}n^3$.

Il costo totale è $O(mn^2)$ per $m \gg n$.

3 Fattorizzazione di Cholesky

3.1 Sfruttare la Simmetria: da LU a $LDL^\top$

La fattorizzazione LU per matrici generali $n \times n$ ha costo $\frac{2}{3}n^3 + O(n^2)$. Per matrici **simmetriche** si può fare molto meglio applicando l'eliminazione in modo simmetrico, producendo la fattorizzazione $LDL^\top$.

Fattorizzazione $LDL^\top$

Per ogni matrice simmetrica $M = M^\top \in \mathbb{R}^{n \times n}$ (per cui l'algoritmo non incontra pivot nulli):

$$M = LDL^\top,$$

dove L è triangolare inferiore con 1 sulla diagonale e D è diagonale. **Costo:** $\frac{1}{3}n^3 + O(n^2)$ — la metà di LU.

3.2 Cholesky per Matrici SPD

Per matrici SPD tutti i pivot sono positivi ($D_{kk} > 0$), quindi $D^{1/2} = \text{diag}(D_{11}^{1/2}, \dots, D_{nn}^{1/2})$ è ben definita. Si ottiene:

Fattorizzazione di Cholesky

Per ogni matrice SPD $M \in \mathbb{R}^{n \times n}$:

$$M = LDL^\top = LD^{1/2}(D^{1/2}L^\top) = R^\top R,$$

dove $R = D^{1/2}L^\top$ è triangolare superiore con elementi diagonali positivi. In MATLAB: `R = chol(A)`, che restituisce R tale che $A = R^\top R$.

Proposizione 3.1 (Stabilità). *Si può dimostrare (tramite la SVD di R) che $\|R\| = \|M\|^{1/2}$. La fattorizzazione di Cholesky è **backward stable** per tutte le matrici SPD e non richiede pivoting.*

Risoluzione di $Mw = b$ con Cholesky

Avendo $M = R^\top R$, il sistema si riduce a due triangolari (forward/backward substitution), ognuna $O(n^2)$:

$$R^\top z = b \Rightarrow z \quad (\text{forward}), \quad Rw = z \Rightarrow w \quad (\text{backward}).$$

Intuizione

Cholesky è il metodo ottimale per sistemi SPD: stabile senza pivoting, costo $\frac{1}{3}n^3$ (metà di LU), ideale per le equazioni normali $A^\top Aw = A^\top y$. Per la regola pratica: “*Is your matrix posdef? Use Cholesky.*”

4 Gradiente Coniugato (CG)

4.1 Motivazione: Sistemi Sparsi di Grandi Dimensioni

Per sistemi SPD di grandi dimensioni e **sparsi** (pochi elementi non nulli), i metodi diretti come Cholesky hanno costo $O(n^3)$ e memoria $O(n^2)$: inapplicabili. Il **Gradiente Coniugato** risolve $Qx = v$ (con $Q \succ 0$) iterativamente, usando *un solo prodotto matrice-vettore per iterazione*.

4.2 Riformulazione come Ottimizzazione

Risolvere $Qx = v$ ($Q \succ 0$) è equivalente a minimizzare:

$$f(x) = \frac{1}{2}x^\top Qx - v^\top x + \text{const.}$$

4.3 Spazi di Krylov e Ottimalità nel Sottospazio

Definizione 4.1 (Spazio di Krylov). Dati $Q \in \mathbb{R}^{m \times m}$ e $v \in \mathbb{R}^m$, lo **spazio di Krylov** di ordine k è:

$$K_k(Q, v) = \text{span}\{v, Qv, Q^2v, \dots, Q^{k-1}v\}.$$

Il CG computa iterati con la **proprietà di ottimalità nel sottospazio**: $x_k = \text{argmin}_{x \in K_k(Q, v)} f(x)$.

4.4 Algoritmo CG e Q -Ortogonalità

Algoritmo: Gradiente Coniugato

```

 $x_0 \leftarrow 0, \quad r_0 \leftarrow d_0 \leftarrow v$ 
for  $j = 1, 2, \dots, m$ :
     $\alpha_j \leftarrow (r_{j-1}^\top r_{j-1}) / (d_{j-1}^\top Q d_{j-1})$  ricerca lineare esatta
     $x_j \leftarrow x_{j-1} + \alpha_j d_{j-1}$ 
     $r_j \leftarrow r_{j-1} - \alpha_j Q d_{j-1}$  aggiornamento residuo
     $\beta_j \leftarrow (r_j^\top r_j) / (r_{j-1}^\top r_{j-1})$ 
     $d_j \leftarrow r_j + \beta_j d_{j-1}$  direzione  $Q$ -ortogonale
end for

```

Proprietà chiave:

- I residui sono ortogonali: $r_i^\top r_j = 0$ per $i \neq j$.
- Le direzioni sono Q -ortogonali: $d_i^\top Q d_j = 0$ per $i \neq j$.
- Convergenza garantita in $\leq m$ iterazioni (aritmetica esatta).
- Costo per iterazione: $O(\text{nnz}(Q))$, un solo prodotto matrice-vettore.
- Storage: solo 3 vettori (x_j, r_j, d_j).

4.5 Convergenza e Numero di Condizionamento

Definizione 4.2 (Numero di Condizionamento). Per $Q \succ 0$:

$$\kappa(Q) = \frac{\lambda_{\max}(Q)}{\lambda_{\min}(Q)} \geq 1.$$

Teorema 4.1 (Convergenza del CG). Dopo k iterazioni:

$$\|x_k - x_*\|_Q \leq 2 \left(\frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1} \right)^k \|x_0 - x_*\|_Q,$$

dove $\|z\|_Q = (z^\top Q z)^{1/2}$ e $\kappa = \kappa(Q)$.

Intuizione

Per ottenere ε -accuratezza nella norma Q : bastano $O(\sqrt{\kappa} \log(1/\varepsilon))$ iterazioni. **Attenzione:** per le equazioni normali $\kappa(A^\top A) = \kappa(A)^2$, quindi il sistema normale è molto più mal condizionato della matrice originale. Il preconditionamento serve a ridurre il κ effettivo.

5 Convessità e Condizioni di Ottimalità

5.1 Definizioni di Convessità

Definizione 5.1 (Convessità). $f : \mathbb{R}^n \rightarrow \mathbb{R}$ è **convessa** se per ogni $x, z \in \mathbb{R}^n$ e $\alpha \in [0, 1]$:

$$f(\alpha x + (1 - \alpha)z) \leq \alpha f(x) + (1 - \alpha)f(z).$$

Equivalentemente (per $f \in C^1$): $f(z) \geq f(x) + \langle \nabla f(x), z - x \rangle$. Equivalentemente (per $f \in C^2$): $\nabla^2 f(x) \succeq 0$.

Definizione 5.2 (Forte Convessità). f è **τ -fortemente convessa** ($\tau > 0$) se:

$$f(z) \geq f(x) + \langle \nabla f(x), z - x \rangle + \frac{\tau}{2} \|z - x\|^2.$$

Per $f \in C^2$: equivale a $\nabla^2 f(x) \succeq \tau I$, ovvero $\lambda_{\min}(\nabla^2 f) \geq \tau > 0$.

Definizione 5.3 (L -Lisciezza). $f \in C^1$ è **L -liscia** se il gradiente è L -Lipschitz:

$$\|\nabla f(x) - \nabla f(z)\| \leq L \|x - z\|, \quad \forall x, z.$$

Per $f \in C^2$: equivale a $\nabla^2 f(x) \preceq LI$, ovvero $\lambda_{\max}(\nabla^2 f) \leq L$.

Minimi quadrati come funzione SPD

Per $f(w) = \frac{1}{2} \|Xw - y\|^2$, l'Hessiano è $\nabla^2 f = X^\top X$. Quindi f è L -liscia con $L = \lambda_{\max}(X^\top X)$ e τ -fortemente convessa con $\tau = \lambda_{\min}(X^\top X)$. Il numero di condizionamento è $\kappa = L/\tau = \kappa(X^\top X)$.

5.2 Convergenza del Metodo del Gradiente con Forte Convessità

Per f L -liscia e τ -fortemente convessa, il metodo del gradiente con passo fisso $\alpha = 2/(L + \tau)$ converge **linearmente**:

$$\|x^{k+1} - x_*\| \leq r^k \|x^1 - x_*\|, \quad r = \frac{\kappa - 1}{\kappa + 1} < 1, \quad \kappa = L/\tau.$$

5.3 Condizioni di Ottimalità per Funzioni Lisce

Per $f \in C^1$ convessa: w^* è minimo globale $\iff \nabla f(w^*) = 0$.

5.4 Sottogradiente e Subdifferenziale

Per funzioni **non differenziabili** (come $\|w\|_1$), il gradiente non esiste in tutti i punti. Si introduce la seguente generalizzazione.

Definizione 5.4 (Sottogradiente). $g \in \mathbb{R}^n$ è un **sottogradiente** di f in x se:

$$f(z) \geq f(x) + \langle g, z - x \rangle, \quad \forall z \in \mathbb{R}^n.$$

A differenza del gradiente, un sottogradiente *non* garantisce la direzione di discesa: può accadere che $f(x - \alpha g) \geq f(x)$ per ogni $\alpha > 0$.

Definizione 5.5 (Subdifferenziale). Il **subdifferenziale** di f in x è l'insieme di tutti i sottogradienti:

$$\partial f(x) = \{g \in \mathbb{R}^n : f(z) \geq f(x) + \langle g, z - x \rangle \forall z\}.$$

Esempio 5.1 (Valore assoluto e norma ℓ_1). Per $f(w) = \|w\|_1$:

$$[\partial f(w)]_i = \begin{cases} \{+1\} & \text{se } w_i > 0, \\ \{-1\} & \text{se } w_i < 0, \\ [-1, +1] & \text{se } w_i = 0. \end{cases}$$

Dove f è differenziabile ($w_i \neq 0$), il subdifferenziale è un singoletto (il gradiente); dove non lo è ($w_i = 0$), è un intervallo chiuso.

Condizione di Ottimalità Generale

Per f convessa (differenziabile o no): w^* è minimo globale se e solo se:

$$0 \in \partial f(w^*).$$

Regola di Somma per il Subdifferenziale

Se $f = f_1 + f_2$ con f_1 differenziabile e f_2 convessa:

$$\partial f(w) = \nabla f_1(w) + \partial f_2(w).$$

Per la funzione obiettivo del progetto $f(w) = \|Xw - y\|^2 + \lambda \|w\|_1$:

$$\partial f(w) = 2X^\top(Xw - y) + \lambda \partial \|w\|_1.$$

6 Ottimizzazione Non Liscia

6.1 Perché i Metodi Lisci Falliscono

Per $f \notin C^1$ come $\|w\|_1$, il metodo del gradiente **non funziona**:

1. Il passo fisso non converge: per $f(x) = L|x|$ con $x^0 = -\alpha L/2$, iterando si ottiene un ciclo $x^0 \rightarrow x^1 \rightarrow x^0 \rightarrow \dots$ senza convergenza.
2. La norma del sottogradiente non è un indicatore di ottimalità: $\|g\|$ piccola non implica $f(x) \approx f_*$, né viceversa. Non c'è un criterio di arresto affidabile.

Complessità dell'Ottimizzazione Non Liscia

Classe	Convessità	Regolarità	Complessità
$f \in C^1$	τ -convessa	L -liscia	$O(\log(1/\varepsilon))$
$f \notin C^1$	τ -convessa	L -Lipschitz	$\Omega(L^2/\varepsilon^2)$
$f \in C^1$	convessa	L -liscia	$O(1/\sqrt{\varepsilon})$
$f \notin C^1$	convessa	L -Lipschitz	$\Omega(L/\varepsilon^2)$

L'ottimizzazione non liscia è **ordini di grandezza più lenta** di quella liscia.

6.2 Metodo del Sottogradiente con Passo DSS

L'iterazione base del metodo del sottogradiente è:

$$x^{i+1} = x^i - \alpha^i g^i, \quad g^i \in \partial f(x^i).$$

Teorema 6.1 (Relazione Fondamentale). Per ogni $g^i \in \partial f(x^i)$:

$$\|x^{i+1} - x_*\|^2 \leq \|x^i - x_*\|^2 + 2\alpha^i(f_* - f(x^i)) + (\alpha^i)^2 \|g^i\|^2.$$

Per α^i piccolo, il termine negativo $2\alpha^i(f_* - f(x^i))$ domina e il passo avvicina a x_* . Il passo non può essere fisso (diverge); deve tendere a zero, ma non troppo in fretta.

Definizione 6.1 (Passo DSS (Diminishing-Square-Summable)).

$$\sum_{i=1}^{\infty} \alpha^i = \infty \quad \text{e} \quad \sum_{i=1}^{\infty} (\alpha^i)^2 < \infty.$$

Esempio: $\alpha^i = 1/i$.

Con il passo DSS: $\exists k$ tale che $f(x^k) - f_* \leq \varepsilon$ (convergenza sulle *iterate record*). Tuttavia la convergenza pratica è molto lenta.

6.3 Passo di Polyak

Se si conosce f_* , si può ottimizzare il passo derivando dalla relazione fondamentale:

$$\alpha_{\text{opt}}^i = \underset{\alpha \geq 0}{\operatorname{argmin}} [\|x^i - x_*\|^2 + 2\alpha(f_* - f^i) + \alpha^2 \|g^i\|^2] = \frac{f(x^i) - f_*}{\|g^i\|^2}.$$

Passo di Polyak (PSS)

$$\alpha^i = \frac{f(x^i) - f_*}{\|g^i\|^2} \geq 0.$$

Con questo passo: $\|x^{i+1} - x_*\|^2 < \|x^i - x_*\|^2$ (monotono decrescente).

Complessità: $O(1/\varepsilon^2)$ sulle *iterate record*, con costante molto migliore di DSS.

Intuizione

Il passo di Polyak è “ottimale” ma richiede la conoscenza di f_* , che in pratica non si ha. L’approccio *target level* stima f_* adattativamente, rifinando la stima se troppo ottimistica.

7 Algoritmo A2: Sottogradient Deflesso con Target Level

7.1 Deflettere la Direzione

Il metodo del sottogradient puro usa $d^i = -g^i$. Il metodo **deflesso** combina il sottogradient corrente con la direzione precedente, analogamente al gradiente coniugato:

$$d^i = \gamma^i g^i + (1 - \gamma^i) d^{i-1}, \quad \gamma^i \in [0, 1].$$

Il passo di aggiornamento è $x^{i+1} = x^i - \alpha^i d^i$.

Intuizione

Si tratta di un “sottogradient coniugato”: sfrutta la memoria delle iterazioni precedenti per costruire una direzione più informata. La direzione deflessa approssima una direzione di discesa migliore, in modo analogo al gradiente coniugato per funzioni quadratiche.

7.2 Scelta Ottimale di γ^i

Il parametro γ^i viene scelto minimizzando la norma quadratica della direzione deflessa:

Formula Chiusa per γ^i

$$\gamma^i = \operatorname{argmin}_{\gamma \in [0,1]} \|\gamma g^i + (1 - \gamma)d^{i-1}\|^2.$$

Svolgendo il quadrato, si ottiene un problema quadratico univariato in γ con soluzione:

$$\gamma_{\text{unc}}^i = \frac{\langle d^{i-1}, d^{i-1} - g^i \rangle}{\|g^i - d^{i-1}\|^2} = \frac{\|d^{i-1}\|^2 - \langle g^i, d^{i-1} \rangle}{\|g^i - d^{i-1}\|^2},$$

e $\gamma^i = \text{clip}(\gamma_{\text{unc}}^i, 0, 1)$.

7.3 Meccanismo Target Level

Il *target level* stima adattativamente f_* per il passo di Polyak. Si mantiene un **valore di riferimento** f_{ref} e una **soglia** $\delta > 0$.

Logica del Target Level

- Il **target level** è $f_{\text{ref}} - \delta$ (livello da raggiungere).
- Il passo di Polyak usa il target al posto di f_* : $\alpha^i = \beta^i(f(x^i) - (f_{\text{ref}} - \delta))/\|d^i\|^2$.
- **Miglioramento buono** ($f(x) \leq f_{\text{ref}} - \delta/2$): si abbassa f_{ref} e si azzerava il contatore r .
- **Troppi passi senza miglioramento** ($r > R$): il target era troppo ambizioso; si riduce $\delta \leftarrow \rho\delta$ (target più facile).

7.4 Algoritmo Completo (A2)

Algoritmo A2: Sottogradient Deflesso con Target Level

Input: $f, x^0, i_{\text{max}}, \beta \in (0, 2), \delta_0 > 0, R > 0, \rho \in (0, 1)$

Output: Valore ottimale approssimato $\bar{f}$

$r \leftarrow 0; \delta \leftarrow \delta_0; f_{\text{ref}} \leftarrow \bar{f} \leftarrow f(x^0); i \leftarrow 1$

while $i < i_{\text{max}}$:

$g^i \in \partial f(x^i)$

calcola un sottogradient

$\gamma^i \leftarrow \operatorname{argmin}_{\gamma \in [0,1]} \|\gamma g^i + (1 - \gamma)d^{i-1}\|^2$

coefficiente di deflazione

$d^i \leftarrow \gamma^i g^i + (1 - \gamma^i)d^{i-1}$

direzione deflessa

$\alpha^i \leftarrow \beta^i \cdot \frac{f(x^i) - (f_{\text{ref}} - \delta)}{\|d^i\|^2}$

passo Polyak con target

$x \leftarrow x^i - \alpha^i d^i$

if $f(x) \leq f_{\text{ref}} - \delta/2$:

buon miglioramento

$f_{\text{ref}} \leftarrow \bar{f}; r \leftarrow 0$

else if $r > R$:

target troppo ambizioso: abbassa soglia

$\delta \leftarrow \rho\delta; r \leftarrow 0$

else:

$r \leftarrow r + \alpha^i \|d^i\|$

accumula progresso

end if

$\bar{f} \leftarrow \min(\bar{f}, f(x)); i \leftarrow i + 1$

end while

7.5 Note sulla Convergenza

Proposizione 7.1. *Sotto condizioni appropriate sui parametri (β^i, γ^i legati dalla condizione “stepsize-restricted” o “deflection-restricted”), la sequenza delle iterate record $\bar{f}^i$ converge a f_* con complessità $O(1/\varepsilon^2)$.*

Il calcolo del sottogradiente $g^i \in \partial f(w^i)$ per $f(w) = \|Xw - y\|^2 + \lambda \|w\|_1$ è:

$$g^i = 2X^\top(Xw^i - y) + \lambda s^i, \quad s_j^i = \begin{cases} +1 & w_j^i > 0 \\ -1 & w_j^i < 0 \\ \in [-1, +1] & w_j^i = 0 \end{cases}$$

8 Algoritmo A1: IRLS (Iteratively Reweighted Least Squares)

8.1 Motivazione: Approssimare ℓ_1 con Minimi Quadrati Pesati

La norma $\|w\|_1 = \sum_i |w_i|$ non è differenziabile. L'idea fondamentale dell'IRLS è di approssimare $|w_i|$ usando il valore dell'iterata precedente w_k :

$$|w_i| = \frac{w_i^2}{|w_i|} \approx \frac{w_i^2}{\max(|w_{k,i}|, \varepsilon_{\text{thr}})}.$$

In questo modo:

$$\|w\|_1 \approx \sum_i \frac{w_i^2}{|w_{k,i}|} = w^\top D_k w = \|W_k w\|^2,$$

dove $W_k = \text{diag}([W_k]_{11}, \dots, [W_k]_{nn})$ con $[W_k]_{ii} = |w_{k,i}|^{-1/2}$.

Intuizione

Questo è un'istanza del framework **Majorization-Minimization (MM)**: ad ogni passo si costruisce una funzione surrogate $\hat{f}_k(w)$ che *maggiora* la funzione originale e la *tocca* nel punto corrente w_k . Minimizzare la surrogate è più semplice e garantisce la discesa monotona della funzione originale.

8.2 Sottoproblema di Minimi Quadrati Pesato

Sostituendo l'approssimazione nell'obiettivo, il sottoproblema all'iterazione k è:

$$w_{k+1} = \underset{w}{\text{argmin}} \|Xw - y\|^2 + 2\lambda \|W_k w\|^2.$$

Espandendo e ponendo il gradiente a zero:

Sistema Lineare dell'IRLS

$$\underbrace{(A + 2\lambda W_k^\top W_k)}_{=: Q_k} w_{k+1} = b, \quad A = X^\top X, \quad b = X^\top y.$$

La matrice Q_k è **SPD** per ogni k (se $\lambda > 0$), quindi si risolve con **fattorizzazione di Cholesky**.

8.3 Aggiornamento della Matrice Peso

$$[W_k]_{ii} = \frac{1}{\sqrt{\max(|w_{k,i}|, \varepsilon_{\text{thr}})}}, \quad i = 1, \dots, n,$$

dove $\varepsilon_{\text{thr}} > 0$ è una soglia per la stabilità numerica (evita divisioni per zero).

Intuizione

Componenti $|w_{k,i}|$ grandi $\Rightarrow [W_k]_{ii}$ piccolo $\Rightarrow$ poca penalizzazione $\Rightarrow$ il componente rimane grande. Componenti $|w_{k,i}|$ piccoli $\Rightarrow [W_k]_{ii}$ grande $\Rightarrow$ forte penalizzazione $\Rightarrow$ il componente viene spinto verso zero. Questo implementa la *sparsità adattiva* tipica della norma ℓ_1 .

8.4 Algoritmo Completo (A1)

Algoritmo A1: IRLS

```

Pre-computazione:  $A \leftarrow X^\top X, \quad b \leftarrow X^\top y$ 

 $w_0 \leftarrow$  soluzione di  $Aw = b$  (minimi quadrati puri, es. con Cholesky)
for  $k = 0, 1, 2, \dots, k_{\max}$ :
    for  $i = 1, \dots, n$ :
         $[W_k]_{ii} \leftarrow \max(|w_{k,i}|, \varepsilon_{\text{thr}})^{-1/2}$  aggiorna pesi
    end for
     $Q_k \leftarrow A + 2\lambda W_k^\top W_k$  matrice SPD
    Risolvi  $Q_k w_{k+1} = b$  con Cholesky sottoproblema LS pesato
    if criterio di arresto soddisfatto: break
end for
return  $w_{k+1}$ 

```

8.5 Convergenza

Proposizione 8.1. Per $\lambda > 0$ e X a rango pieno, l'IRLS converge. La convergenza è **lineare (Q-lineare)** in un intorno della soluzione w_* :

$$\|w_{k+1} - w_*\| \leq r \|w_k - w_*\|, \quad r < 1.$$

Il tasso r dipende dalla struttura della soluzione (grado di sparsità di w_*).

Osservazione 8.1. Il costo per iterazione è $O(n^2)$ per aggiornare W_k e Q_k , più $O(n^3/3)$ per la fattorizzazione di Cholesky. Nella pratica, il numero di iterazioni esterne è piccolo (convergenza rapida vicino alla soluzione) e IRLS risulta molto efficiente.

9 Riepilogo: Connessioni tra gli Argomenti

Questo manuale copre gli argomenti necessari per implementare e comprendere il Progetto 25, organizzati nel seguente schema di dipendenze:

Argomento	Sezione	Uso nel Progetto
Minimi Quadrati	2	Formulazione base ELM, sottoproblemi IRLS
Equazioni Normali	2	$A^\top Aw = A^\top y$, struttura di Q_k in A1
Cholesky	3	Solver interno di A1 per ogni iterazione
Gradiente Coniugato	4	Alternativa iterativa per sistemi sparsi
Convessità / L -lisciezza	5	Garanzie di ottimalità globale
Subdifferenziale	5	Condizioni di ottimalità per LASSO
Complessità non liscia	6	Perché servono metodi speciali
Passo di Polyak	6	Stepsize ottimale in A2
Target Level	7	Stima adattiva di f_* in A2
Direzione Deflessa	7	Direzione migliorata in A2
MM Framework	8	Giustificazione teorica di IRLS

Percorso di Studio Consigliato

Sezione 1 (ELM + LASSO) $\rightarrow$ Sezione 2 (minimi quadrati) $\rightarrow$ Sezione 3 (Cholesky)
 $\rightarrow$ Sezione 4 (CG) $\rightarrow$ Sezione 5 (convessità) $\rightarrow$ Sezione 6 (ottimizzazione non liscia)
 $\rightarrow$ Sezione 8 (IRLS) $\rightarrow$ Sezione 7 (Sottogradiente Deflesso).